

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities. (BL5, BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

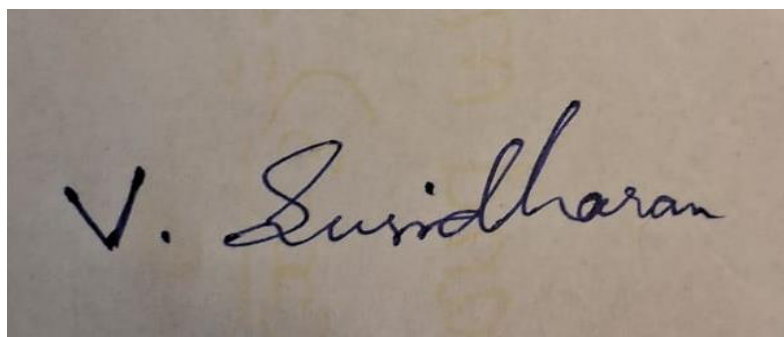
PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

I susidharan(192521387) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

A photograph of a handwritten signature in black ink on a light-colored, slightly textured paper. The signature is written in a cursive style and reads "V. Susidharan". The first letter 'V' is large and prominent, followed by a period and the name "Susidharan".

Question 1: Stateful Firewall System

```
STRUCTURE ConnectionEntry:
    src_ip, src_port, dst_ip, dst_port, protocol
    state          // NEW, ESTABLISHING, ESTABLISHED, CLOSING
    last_seen_time

ConnectionTable = empty hash map    // key = 5-tuple

FUNCTION ProcessPacket(packet):
    key = (packet.src_ip, packet.src_port, packet.dst_ip, packet.dst_port, packet.protocol)
    reverse_key = (packet.dst_ip, packet.dst_port, packet.src_ip, packet.src_port,
packet.protocol)

    IF key EXISTS in ConnectionTable OR reverse_key EXISTS in ConnectionTable:
        entry = ConnectionTable[key or reverse_key]
        UpdateState(entry, packet)
        entry.last_seen_time = current_time()
        RETURN ALLOW                // fast path: part of a known session

    ELSE:
        IF packet.protocol == TCP AND packet.flags != SYN:
            RETURN DENY              // mid-stream packet, no matching session

        decision = EvaluateStaticRules(packet)
        IF decision == ALLOW:
            ConnectionTable[key] = ConnectionEntry(state=NEW, last_seen_time=current_time())
            RETURN ALLOW
        ELSE:
            RETURN DENY

FUNCTION UpdateState(entry, packet):
    IF packet.flags CONTAINS FIN OR RST:
        entry.state = CLOSING
    ELSE IF packet.flags CONTAINS SYN AND ACK:
        entry.state = ESTABLISHING
    ELSE IF entry.state == ESTABLISHING AND packet.flags CONTAINS ACK:
        entry.state = ESTABLISHED

FUNCTION PeriodicCleanup():          // run every T seconds
    FOR EACH entry IN ConnectionTable:
        IF current_time() - entry.last_seen_time > TIMEOUT:
            REMOVE entry FROM ConnectionTable
```

Evaluation: state tracking vs. stateless filtering

Stateless filtering evaluates every packet independently against static rules with no memory of prior traffic. This forces administrators to explicitly open wide ranges of return-traffic ports and cannot distinguish a genuinely established session from an unsolicited packet merely claiming to belong to one (e.g., a bare ACK sent with no prior SYN), leaving it vulnerable to spoofed or scanning traffic that mimics legitimate flows.

Stateful filtering closes this gap: ProcessPacket only allows a non-SYN packet through if it matches an entry already created by a legitimate handshake, so an attacker cannot inject mid-stream traffic without a tracked session. This meaningfully improves security (blocking spoofing and unsolicited connection attempts) and reduces the number of ports that must be left open, at the cost of the memory/compute needed to maintain the connection table —

which is why PeriodicCleanup and timeout logic are essential to prevent table exhaustion under high connection volume or an attempted resource-exhaustion attack.

Question 2: Signature-Based Malware Detection

```
SignatureDatabase = LoadSignatures("signatures.db") // known malware hashes + byte patterns

FUNCTION ScanFile(file_path):
    file_data = ReadFile(file_path)
    file_hash = ComputeHash(file_data)                // e.g., SHA-256

    IF file_hash IN SignatureDatabase.hash_set:
        RETURN INFECTED, SignatureDatabase.hash_set[file_hash]

    FOR EACH signature IN SignatureDatabase.byte_patterns: // catches simple variants
        IF ContainsPattern(file_data, signature.pattern):
            RETURN INFECTED, signature.name

    RETURN CLEAN, null

FUNCTION ContainsPattern(data, pattern):
    RETURN (pattern FOUND WITHIN data) // e.g., Aho-Corasick for many patterns at once

FUNCTION ScanDirectory(root_path):
    results = empty list
    FOR EACH file IN RecursivelyListFiles(root_path):
        status, match = ScanFile(file)
        IF status == INFECTED:
            results.append({file, match})
            QuarantineFile(file)
    RETURN results

FUNCTION UpdateSignatureDatabase():
    new_signatures = FetchFromThreatIntelligenceFeed()
    SignatureDatabase = Merge(SignatureDatabase, new_signatures)
```

Evaluation: effectiveness and limitations

This approach is highly effective and efficient against any threat already represented in the database: hash lookup is near $O(1)$ and pattern search is fast, giving reliable, low-false-positive detection of known malware with minimal runtime overhead.

Its fundamental limitation is that it can only detect what it already knows: it is completely blind to zero-day malware, and even trivial modification of known malware defeats it — a single changed byte alters the hash entirely, and basic packing/obfuscation/polymorphism can defeat byte-pattern signatures too. Detection therefore depends entirely on the database being current, leaving a real exposure window between a new threat's release and the next signature update, which is why signature-based detection is normally layered alongside behavioral/anomaly-based detection rather than relied on alone.

Question 3: DDoS Detection via Traffic Threshold Analysis

```
BASELINE_WINDOW = 24 hours
CURRENT_WINDOW = 60 seconds
THRESHOLD_MULTIPLIER = 3
SYN_RATIO_THRESHOLD = 0.8
```

```

FUNCTION ComputeBaseline(historical_traffic):
    baseline_rate = Average(historical_traffic.packets_per_second, over BASELINE_WINDOW)
    baseline_stddev = StdDev(historical_traffic.packets_per_second, over BASELINE_WINDOW)
    RETURN baseline_rate, baseline_stddev

FUNCTION MonitorTraffic():
    baseline_rate, baseline_stddev = ComputeBaseline(HistoricalLog)

    WHILE true:
        current_rate = MeasurePacketsPerSecond(CURRENT_WINDOW)
        source_ip_diversity = CountUniqueSourceIPs(CURRENT_WINDOW)
        syn_ratio = CountSYNOnly(CURRENT_WINDOW) / CountTotalTCP(CURRENT_WINDOW)

        is_volume_anomaly = current_rate > (baseline_rate + THRESHOLD_MULTIPLIER *
baseline_stddev)
        is_handshake_anomaly = syn_ratio > SYN_RATIO_THRESHOLD
        is_source_anomaly = source_ip_diversity DIFFERS ABNORMALLY FROM
ExpectedDiversity(baseline_rate)

        IF is_volume_anomaly AND (is_handshake_anomaly OR is_source_anomaly):
            RaiseAlert("Possible DDoS attack detected")
            TriggerMitigation()
        ELSE IF is_volume_anomaly:
            LogEvent("High traffic volume -- monitoring, not yet confirmed as attack")

        Sleep(CURRENT_WINDOW)

FUNCTION TriggerMitigation():
    ApplyRateLimiting(suspect_sources)
    NotifyUpstreamScrubbingService()

```

Evaluation: differentiating legitimate traffic from attack traffic

The algorithm deliberately avoids alerting on volume alone, since a legitimate flash crowd (viral content, a sale event) can spike traffic just as much as an attack. Instead it requires a volume anomaly AND a supporting structural signal: an abnormally high ratio of incomplete TCP handshakes (real browsers reliably complete the handshake; SYN-flood traffic often doesn't), or a source-IP distribution inconsistent with organic growth (either implausibly concentrated in a few sources, as in reflection/amplification attacks, or implausibly large/dispersed in a way real user growth doesn't produce). Requiring both dimensions together, rather than a single volume threshold, substantially reduces false alarms during genuine legitimate traffic surges while still catching the behavioral fingerprints characteristic of actual DDoS traffic.

Question 4: Network Intrusion Detection using Machine Learning Classification

```

FUNCTION ExtractFeatures(network_flow):
    features = {
        duration: flow.end_time - flow.start_time,
        packet_count: flow.packet_count,
        byte_count: flow.total_bytes,
        avg_packet_size: flow.total_bytes / flow.packet_count,
        flag_distribution: CountTCPFlags(flow),
        dst_port: flow.dst_port,
        protocol: flow.protocol,
        inter_arrival_variance: Variance(flow.packet_timestamps)
    }

```

```

    }
    RETURN NormalizeFeatures(features)

FUNCTION TrainModel(labeled_training_data):
    X = [ExtractFeatures(flow) for flow in labeled_training_data]
    y = [flow.label for flow in labeled_training_data]    // "normal", "DoS", "probe", "R2L", ...

    X_train, X_val, y_train, y_val = SplitTrainValidation(X, y)
    model = InitializeClassifier()           // e.g., Random Forest / SVM / Neural Network
    model.fit(X_train, y_train)
    accuracy = Evaluate(model, X_val, y_val)
    RETURN model, accuracy

FUNCTION ClassifyLiveTraffic(model, live_flow):
    features = ExtractFeatures(live_flow)
    prediction = model.predict(features)
    confidence = model.predict_proba(features)

    IF prediction != "normal" AND confidence > ALERT_THRESHOLD:
        RaiseAlert(live_flow, prediction, confidence)
    RETURN prediction

FUNCTION RetrainPeriodically(model, new_labeled_data):
    updated_model, accuracy = TrainModel(CombinedDataset(existing_data, new_labeled_data))
    RETURN updated_model

```

Evaluation: impact of feature selection and training data

Feature selection sets a hard ceiling on what the model can possibly learn: omitting a discriminative feature (e.g., TCP flag distribution, which is highly informative for identifying SYN-flood/probe patterns) caps achievable accuracy regardless of the classifier chosen, while irrelevant or noisy features dilute the learnable signal and increase overfitting risk, especially with a limited training set.

Training data quality and representativeness matter just as much: a model trained on data that doesn't cover the full range of legitimate traffic will misclassify unusual-but-benign flows as malicious (false positives), while training data with limited or dated attack examples fails to generalize to new attack variants (false negatives). This is why RetrainPeriodically matters — incorporating newly confirmed data keeps the model current against both evolving legitimate usage patterns and evolving attacker techniques (concept drift), and the training set needs balanced, diverse examples across normal traffic and every attack category to avoid bias toward whichever class was overrepresented during training.

Question 5: Secure Network Access Control with Role-Based Access

```

Users = {}           // user_id -> {password_hash, roles[]}
Roles = {}           // role_name -> {permissions[]}
ActiveSessions = {}  // session_token -> {user_id, roles[], expiry_time}

FUNCTION Authenticate(user_id, password_attempt):
    IF user_id NOT IN Users:
        RETURN FAILURE
    IF Hash(password_attempt) == Users[user_id].password_hash:
        // optionally: AND VerifyMFA(user_id) for a second factor
        session_token = GenerateSecureToken()
        ActiveSessions[session_token] = {
            user_id: user_id,
            roles: Users[user_id].roles,
            expiry_time: current_time() + SESSION_DURATION

```

```

    }
    RETURN SUCCESS, session_token
ELSE:
    LogFailedAttempt(user_id)
    RETURN FAILURE

FUNCTION AuthorizeAccess(session_token, requested_resource, requested_action):
    IF session_token NOT IN ActiveSessions:
        RETURN DENY
    session = ActiveSessions[session_token]
    IF current_time() > session.expiry_time:
        REMOVE ActiveSessions[session_token]
        RETURN DENY

    FOR EACH role IN session.roles:
        IF (requested_resource, requested_action) IN Roles[role].permissions:
            LogAccess(session.user_id, requested_resource, requested_action, ALLOWED)
            RETURN ALLOW

    LogAccess(session.user_id, requested_resource, requested_action, DENIED)
    RETURN DENY

FUNCTION AssignRole(admin_session, target_user_id, role_name):
    IF NOT AuthorizeAccess(admin_session, "role_management", "assign"):
        RETURN DENY // only authorized admins can assign roles
    Users[target_user_id].roles.append(role_name)
    RETURN SUCCESS

```

Evaluation: how access control enhances network security

- Least privilege: each user receives only the permissions tied to their assigned role(s), so even a compromised regular-user account limits an attacker to that narrow permission set rather than the whole network.
- Continuous authorization: separating authentication (identity) from authorization (per-request permission checks) means every resource request is checked against current session validity and role permissions, so a revoked role or expired session is enforced immediately rather than relying on a one-time login decision.
- Accountability: centralized logging of both allowed and denied access attempts provides an audit trail supporting detection of misuse, insider threats, or compromised-credential abuse after the fact.

Together, this structured, centrally managed permission model reduces the overall attack surface, contains the blast radius of any single compromised account, and provides accountability — protections that are significantly weaker or entirely absent in unrestricted or ungoverned access models.